BigQuery Cheat Sheet

Partitioning & Clustering

1. Partitioning

Splits a table into segments (partitions) based on a column. Queries that filter on the partition column scan only the matching partitions (**partition pruning**), reducing cost and time.

Types of partitioning

Type	Based on	Notes
Time-unit column	DATE, TIMESTAMP or DATETIME column	Granularity: HOUR, DAY, MONTH, YEAR. Most common choice.
Ingestion time	Load/arrival time (_PARTITIONTIME, _PARTITIONDATE)	Use when data has no good date column.
Integer range	INT64 column	Define start, end and interval (e.g. customer_id 0-1000 step 10).

Example DDL

```
CREATE TABLE mydataset.orders (
  order_id INT64, customer_id INT64, order_ts TIMESTAMP, amount NUMERIC)
PARTITION BY DATE(order_ts)
OPTIONS (
  partition_expiration_days = 365,
  require_partition_filter = TRUE);

-- Integer range
PARTITION BY RANGE_BUCKET(customer_id, GENERATE_ARRAY(0, 1000, 10))

-- Ingestion time (daily)
PARTITION BY _PARTITIONDATE
```

Key facts

- Only **one** partition column per table; it must be top-level and not repeated (no ARRAY).
- Max **10,000 partitions** per table.
- Special partitions: __**NULL**__ (NULL values) and __**UNPARTITIONED**__ (out-of-range/invalid values).
- **require_partition_filter** forces queries to filter on the partition column (prevents costly full scans).
- **partition_expiration_days** auto-deletes old partitions.
- Cost is known before running (dry run shows exact bytes for pruned partitions).
- Partitions unmodified for 90 days switch to cheaper long-term storage pricing individually.
- Prune by filtering on the raw column: `WHERE order_ts >= '2026-01-01'`. Avoid wrapping it in expressions or subqueries that hide the filter.
- Best when partitions are reasonably large (roughly 10 GB+ each); too many tiny partitions hurts performance.

2. Clustering

Sorts and co-locates rows in storage blocks based on the values of up to **4 columns**. BigQuery skips blocks that cannot match your filter (**block pruning**).

Example DDL

```
CREATE TABLE mydataset.orders_clustered (
  order_id INT64, customer_id INT64, country STRING, order_ts TIMESTAMP)
PARTITION BY DATE(order_ts)
CLUSTER BY country, customer_id;   -- order matters!
```

Key facts

- Up to **4 clustering columns**; **column order matters** (filter on the first column gives the most benefit; filters on later columns alone help far less).
- Supported types: STRING, INT64, NUMERIC, BIGNUMERIC, BOOL, DATE, DATETIME, TIMESTAMP, GEOGRAPHY. **Not** supported: FLOAT64, JSON, ARRAY, STRUCT.
- Works with or without partitioning. Usually best **combined**: partition by date, cluster by high-cardinality filter columns.
- **Auto-reclustering** runs in the background at no extra cost as new data arrives.
- Benefits filters (=, <, >, BETWEEN, IN, prefix LIKE), aggregations and some joins on clustered columns.
- Cost estimate before running is an **upper bound**; actual bytes billed may be lower.
- Most useful for larger tables (about 1 GB+). Little gain on small tables.
- Good clustering columns: frequently filtered/joined, high cardinality (user_id, country, product_id).

3. Partitioning vs Clustering

Feature	Partitioning	Clustering
Granularity	Whole partitions	Storage blocks within a table/partition
Columns	1	Up to 4 (ordered)
Cost estimate	Exact before query	Upper bound only
Cardinality	Low to moderate (max 10,000 partitions)	High cardinality is fine
Data types	DATE/TIMESTAMP/DATETIME, INT64, ingestion time	STRING, INT64, DATE, BOOL, TIMESTAMP, etc.
Maintenance	Partition-level expiration, management	Automatic reclustering
Filter enforcement	require_partition_filter option	None
Best for	Time-based filtering, data lifecycle control	Filtering on many or high-cardinality columns

4. When to Use What

- Queries almost always filter by date/time: **partition** by that column.
- Also filter by user, country, product, status, etc.: add **clustering** on those columns.
- Need predictable cost per query, or data retention rules: **partition** + require_partition_filter.
- Column has too many distinct values for partitions (e.g. user_id with millions): **cluster** instead.
- Partitions would be tiny (under ~1 GB): prefer clustering alone, or coarser partitions (monthly).

5. Useful Commands

```
-- Inspect partitions
SELECT * FROM mydataset.INFORMATION_SCHEMA.PARTITIONS
WHERE table_name = 'orders';

-- Change settings on an existing table
ALTER TABLE mydataset.orders SET OPTIONS (require_partition_filter = TRUE);

-- Convert an unpartitioned table (create a new one)
CREATE TABLE mydataset.new_tbl
PARTITION BY DATE(order_ts) CLUSTER BY customer_id
AS SELECT * FROM mydataset.old_tbl;
```

Tip: Always run a dry run (or check the bytes estimate in the console) to confirm pruning is working. Partitioning and clustering settings on an existing table generally can't be changed in place; recreate the table.